

中山大學
SUN YAT-SEN UNIVERSITY

数字图像处理—第四章作业

快速傅里叶变换与频域滤波实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 骆伟祺

起 始 日 期: 2025年12月9日

结 束 日 期: 2025年12月10日

实 验 地 点: B205

目录

1 实验简述 **2**

2 理论分析 **2**

3 代码实现 **2**

4 结果展示 **3**

5 实验总结 **5**

1　实验简述

本实验要求实现基本形态学运算，包括二值图像的膨胀和腐蚀，并利用腐蚀结果进行形态学边界提取。具体任务为：编写程序，支持任意指定的3×3结构元素，对二值图像进行膨胀和腐蚀操作，并根据公式 $\beta(A) = A - (A \ominus B)$ 提取字符边界。实验对象为1.png中的“UTK”字符图片。

2　理论分析

形态学运算是图像处理中的重要工具，主要包括膨胀和腐蚀：

- **腐蚀(Erosion)**：$A \ominus B = \{z | B_z \subseteq A\}$，即结构元素$B$在点$z$处完全包含于前景$A$时，输出为1，否则为0。腐蚀可使前景区域收缩，去除边界像素。
- **膨胀(Dilation)**：$A \oplus B = \{z | (B^s)_z \cap A \neq \emptyset\}$，即结构元素$B$在点$z$处与前景$A$有交集时，输出为1。膨胀可使前景区域扩展，填补小孔洞。
- **边界提取(Boundary Extraction)**：$\beta(A) = A - (A \ominus B)$，即原图减去腐蚀结果，得到单像素宽度的边界。

结构元素B为3×3二值矩阵，图像边界采用0填充。

3　代码实现

本实验核心代码如下，支持任意指定的3×3结构元素。可通过参数灵活切换结构元素类型，分别进行膨胀、腐蚀和边界提取。

结构元素示例：

```
# 3全结构元素x31
se_full = np.ones((3, 3), dtype=np.uint8)
# 3十字结构元素x3
se_cross = np.array([[0,1,0],[1,1,1],[0,1,0]], dtype=np.uint8)
```

形态学操作类：

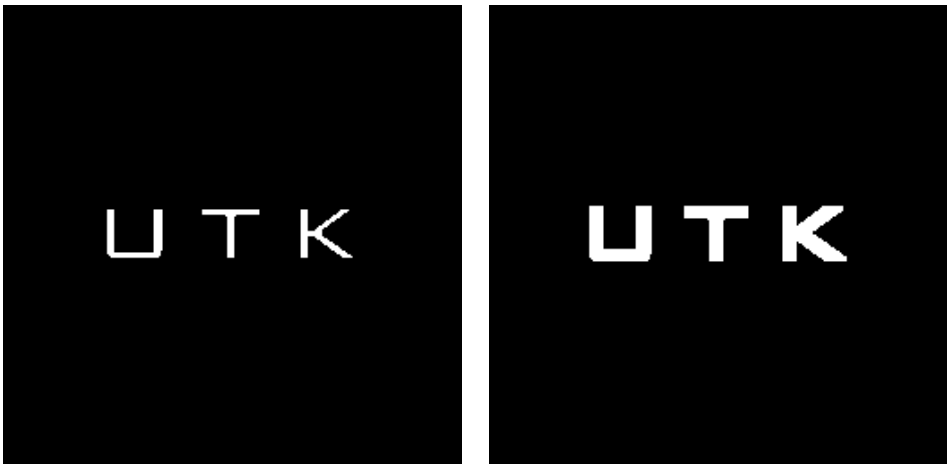
```
class MorphologicalOperations:
    def __init__(self, structure_element=None):
        # 支持任意3结构元素x3
        if structure_element is None:
            self.SE = np.ones((3, 3), dtype=np.uint8)
        else:
            self.SE = np.array(structure_element, dtype=np.uint8)
        if self.SE.shape != (3, 3):
            raise ValueError("结构元素必须是3矩阵x3")
    def _pad_image(self, image, padding_value=0):
        # 图像边界填充
        h, w = image.shape
        padded = np.full((h+2, w+2), padding_value, dtype=image.dtype)
        padded[1:h+1, 1:w+1] = image
        return padded
    def erosion(self, binary_image):
        # 腐蚀运算
        h, w = binary_image.shape
        padded = self._pad_image(binary_image, 0)
```

```
        output = np.zeros_like(binary_image)
        for i in range(h):
            for j in range(w):
                window = padded[i:i+3, j:j+3]
                if np.all(window[self.SE == 1] == 1):
                    output[i, j] = 1
        return output
    def dilation(self, binary_image):
        # 膨胀运算
        h, w = binary_image.shape
        padded = self._pad_image(binary_image, 0)
        output = np.zeros_like(binary_image)
        for i in range(h):
            for j in range(w):
                window = padded[i:i+3, j:j+3]
                if np.any((window == 1) & (self.SE == 1)):
                    output[i, j] = 1
        return output
    def boundary_extraction(self, binary_image):
        # 边界提取
        eroded = self.erosion(binary_image)
        boundary = binary_image.astype(int) - eroded.astype(int)
        boundary = np.clip(boundary, 0, 1)
        return boundary.astype(np.uint8)
```

上述代码实现了对同一张图片分别用两种结构元素做膨胀、腐蚀和边界提取，并保存各自结果。结构元素可灵活指定，满足实验要求。

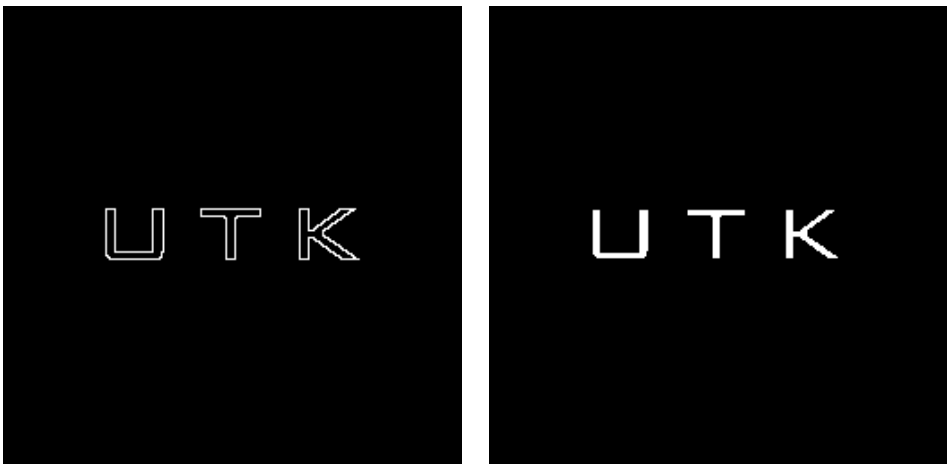
4　结果展示

分别采用两种结构元素（3x3全1和3x3十字形）对1.png进行膨胀、腐蚀和边界提取，结果：



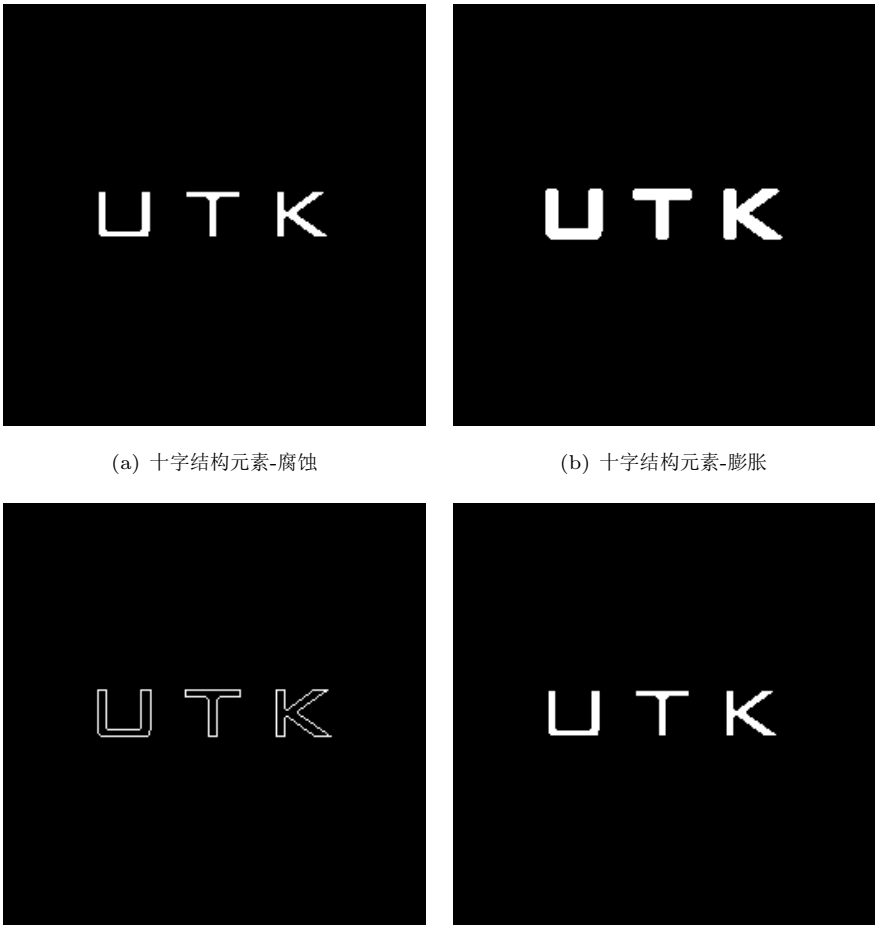
(a) 全1结构元素-腐蚀　　(b) 全1结构元素-膨胀

图 1: 结构元素1（3x3全1）腐蚀与膨胀结果



(a) 全1结构元素-边界　　(b) 全1结构元素-原图

图 2: 结构元素1（3x3全1）边界提取与原图



(a) 十字结构元素-腐蚀　　(b) 十字结构元素-膨胀

(c) 十字结构元素-边界　　(d) 十字结构元素-原图

图 3: 结构元素2（3x3十字形）处理结果

对比两种结构元素的处理效果如下：

- **腐蚀效果：** 全1结构元素对字符边界的腐蚀更为均匀，字符整体变细，细小的笔画和角落容

易被完全腐蚀掉，适合整体收缩。十字结构元素则对水平方向和垂直方向的腐蚀更明显，对角线方向的细节保留较多，字符骨架更突出。

- **膨胀效果：** 全1结构元素膨胀后字符整体变粗，边界向各个方向均匀扩展，容易造成笔画粘连。十字结构元素膨胀时主要在水平和垂直方向扩展，字符的角和斜线部分扩展较少，结构更清晰。
- **边界提取效果：** 全1结构元素提取的边界较为完整，轮廓线条较粗，部分细节可能被覆盖。十字结构元素提取的边界更细腻，线条更细，字符的细节和骨架结构表现更明显，适合需要保留字符细节的场景。
- **适用场景：** 全1结构元素适合整体轮廓提取和粗略边界检测，十字结构元素适合细节分析和骨架提取。实际应用中可根据目标选择合适的结构元素。

5　实验总结

本实验实现了二值图像的膨胀、腐蚀及形态学边界提取，支持任意3×3结构元素。通过对“UTK”字符图片分别采用全1和十字形结构元素处理，发现结构元素的选择会影响边界检测的效果。全1结构元素适合整体轮廓提取，十字形结构元素适合细节和方向性分析。实验加深了对形态学基本原理和算法实现的理解，为后续图像分析任务打下基础。